ML Metrics & Fundamental Concepts Guide

Complete Reference with Analogies

PART 1 — FUNDAMENTAL CONCEPTS

🎯 Bias vs Variance — The Core Tradeoff

What They Are

Bias = Error from wrong assumptions in the learning algorithm

- High bias = model is too simple, underfits
- Model makes systematic errors, consistently misses the pattern

Variance = Error from sensitivity to small fluctuations in training data

- High variance = model is too complex, overfits
- Model memorizes training data, performs poorly on new data

💡 Analogy: Archer Shooting at a Target

- **Low Bias** = arrows hit near the bullseye on average
- **High Bias** = arrows consistently miss in one direction
- **Low Variance** = arrows cluster tightly together
- **High Variance** = arrows scattered all over

Your goal: Hit the bullseye consistently (low bias) with tight grouping (low variance)

💡 Another Analogy: Thermometer Accuracy

High Bias Thermometer:

> Always reads 5°C higher than actual temperature. Consistent but wrong. Like a model that always underpredicts or overpredicts systematically.

High Variance Thermometer:

> Sometimes reads correctly, sometimes wildly wrong — depends on where you hold it, room draft, etc. Like a model that changes drastically with small data changes.

Perfect Thermometer:

> Always reads the correct temperature regardless of conditions. Low bias (accurate) + low variance (consistent).

The Bias-Variance Tradeoff

```
Model Complexity →

Simple Model                Complex Model
(Linear)                  (Deep Neural Net)

High Bias        ←→        Low Bias
Low Variance     ←→        High Variance
Underfits        ←→        Overfits

          Sweet Spot
              ↓
        Balanced Model
     (Random Forest, XGBoost)
```

The tradeoff: You can't minimize both simultaneously

- Simplify model → reduce variance, increase bias
- Complexify model → reduce bias, increase variance

Real Examples by Model

Model	Bias	Variance	Why
Linear Regression	High	Low	Too simple, assumes linear relationship
Decision Tree (deep)	Low	High	Memorizes training data
Random Forest	Low	Low	Averages many trees, reduces variance
kNN (k=1)	Low	High	Memorizes, sensitive to noise
kNN (k=100)	High	Low	Too smooth, misses patterns
XGBoost (tuned)	Low	Low	Sequential correction reduces bias, regularization reduces variance

---

How to Diagnose

High Bias (Underfitting):

- Training accuracy: Low
- Test accuracy: Low (similar to train)
- **Fix:** Use more complex model, add features, reduce regularization

High Variance (Overfitting):

- Training accuracy: Very high
- Test accuracy: Much lower
- **Fix:** Get more data, use simpler model, increase regularization, cross-validation

💡 **Analogy:** Studying for an exam

- High Bias = didn't study enough, fail both practice tests and real exam
- High Variance = memorized practice tests word-for-word, ace practice but fail real exam
- Sweet Spot = understood concepts, ace both practice and real exam

---

🎯 Overfitting vs Underfitting

Underfitting

What: Model is too simple to capture the underlying pattern **Sign:** Poor performance on both training and test data **Cause:** Model has high bias

💡 **Analogy:** Using a ruler to draw a curved road on a map

> The road curves and winds, but you can only draw straight lines. Your map (model) is too simple — it doesn't fit the actual road (data). Both people who helped you (training data) and new travelers (test data) get lost.

Overfitting

What: Model memorizes training data including noise **Sign:** Excellent training performance, poor test performance **Cause:** Model has high variance

💡 **Analogy:** Student who memorizes answers instead of learning concepts

> Student memorizes "Question 5 answer is B, Question 7 answer is C" from practice exam. Aces practice exam (training data) but fails real exam (test data) because questions are slightly different. They memorized specific examples, not the underlying concept.

Another Analogy: Tailor-made suit

> A suit perfectly tailored to your exact measurements fits YOU perfectly but won't fit your brother even though he's similar size. The suit "overfitted" to you — it captured not just your general size but also your unique quirks. A "generalized" suit (like a size M) fits reasonably well for many people.

The Sweet Spot — Just Right

💡 **Analogy:** Goldilocks and the Three Bears

- Papa Bear's model: Too simple (underfit) — straight line through curved data
- Baby Bear's model: Too complex (overfit) — wiggly line through every single point including noise
- Mama Bear's model: Just right — captures the true pattern, ignores noise

---

🎯 Regularization (L1 & L2)

What is Regularization?

Adding a penalty to large weights/coefficients to prevent overfitting.

💡 **Analogy:** Speed limit on a highway

> Without speed limits (no regularization), some drivers go 200 mph (weights explode), causing crashes (overfitting). Speed limits (regularization) keep everyone at reasonable speeds (bounded weights), making the road safer (better generalization).

L1 Regularization (Lasso)

Formula: Loss + λ × Σ|weights| **Effect:** Drives some weights to exactly zero → feature selection

💡 **Analogy:** Marie Kondo-ing your closet

> You go through your clothes asking "Does this spark joy?" (Is this feature useful?). If not → throw it away (weight = 0). You end up with fewer items (features) but only the ones you actually wear (important features). Your closet (model) is simpler and cleaner.

When to use: When you have many features and suspect most are irrelevant

---

L2 Regularization (Ridge)

Formula: Loss + λ × Σ(weights²) **Effect:** Shrinks all weights evenly, none reach exactly zero

💡 **Analogy:** Turning down the volume on all instruments in a band

> Every instrument (feature) plays slightly quieter (smaller weight), but they all still play. No instrument is silenced completely. The music (prediction) becomes more balanced — no single instrument drowns out others.

When to use: When all features are somewhat useful, you just want to prevent any from dominating

---

L1 vs L2 Visual Comparison

```
No Regularization:
weights = [50, -30, 45, 28, -40]  → Some weights huge

L2 Regularization:
weights = [15, -10, 14, 9, -13]   → All shrunk evenly

L1 Regularization:
weights = [22, 0, 18, 0, -20]     → Sparse, some zeroed out
```

🎯 Train-Test Split & Cross-Validation

Train-Test Split

What: Split data into training set (80%) and test set (20%) **Why:** Test set simulates unseen data to evaluate generalization

💡 **Analogy:** Practice exam vs Real exam

> You study using practice questions (training set) and test yourself on a separate final exam (test set) you've never seen. If you use final exam questions to study, you're cheating yourself — you won't know if you truly understand or just memorized.

Critical Rule: NEVER let the model see test data during training

- Don't fit preprocessors (scaler, encoder) on test data
- Don't tune hyperparameters on test data
- Test set must remain completely unseen until final evaluation

---

Cross-Validation (k-Fold)

What: Split data into k parts, train on k-1, test on 1, repeat k times, average results

💡 **Analogy:** Rotating group presentations

> Class of 25 students split into 5 groups of 5. Each week, 1 group presents while 4 groups evaluate them. After 5 weeks, every group has presented once and evaluated 4 times. Now you have a fair average score for each group based on multiple evaluators.

```
Fold 1:  [Test] [Train] [Train] [Train] [Train]
Fold 2:  [Train] [Test] [Train] [Train] [Train]
Fold 3:  [Train] [Train] [Test] [Train] [Train]
Fold 4:  [Train] [Train] [Train] [Test] [Train]
Fold 5:  [Train] [Train] [Train] [Train] [Test]

Final score = average of all 5 test scores
```

Why better than single train-test split:

- More reliable estimate (less dependent on lucky/unlucky split)
- Uses all data for both training and testing
- Reduces variance in performance estimate

When to use:

- Small datasets (maximize use of limited data)
- Hyperparameter tuning
- Model selection

🎯 Feature Scaling — Why & When

Why Scale Features?

Distance-based algorithms measure "closeness" — features on larger scales dominate.

💡 **Analogy:** Measuring similarity between people using age and income

```
Person A: Age 25, Income $40,000
Person B: Age 30, Income $45,000
Person C: Age 26, Income $100,000

Without scaling:
Distance(A, B) = √[(30-25)² + (45000-40000)²] = 5000.0025 ← income dominates
Distance(A, C) = √[(26-25)² + (100000-40000)²] = 60000.001

With scaling (0-1):
A: [0.0, 0.0]    B: [1.0, 0.083]    C: [0.2, 1.0]
Distance(A, B) = 1.003
Distance(A, C) = 1.02

Now age matters! Without scaling, age difference was invisible.
```

When to Scale

MUST scale:

- kNN (distance-based)
- SVM (distance-based)
- Neural Networks (gradient descent works better)
- PCA (variance-based)
- Logistic Regression, Linear Regression (helps gradient descent converge faster)

DON'T need to scale:

- Decision Trees (uses splits, not distances)
- Random Forest (ensemble of trees)
- XGBoost / Gradient Boosting (tree-based)

Types of Scaling

StandardScaler (Z-score normalization)

```python
(x - mean) / std_dev
# Result: mean=0, std=1
```

💡 **Analogy:** Grading on a curve

> Convert everyone's raw scores to "how many standard deviations above/below average". A score of +2 means you're 2 std devs above class average.

MinMaxScaler

```python
(x - min) / (max - min)
# Result: all values between 0 and 1
```

💡 **Analogy:** Normalizing test scores

> Hardest question = 1.0, easiest = 0.0. Your score on each question shows how close you got to the max possible points.

Use StandardScaler by default unless you specifically need bounded [0,1] range

---

PART 2 — CLASSIFICATION METRICS

🎯 Confusion Matrix — Foundation of Everything

The 2x2 Grid

```
              Predicted
              Pos    Neg
Actual  Pos  | TP  | FN  |
        Neg  | FP  | TN  |
```

TP (True Positive): Correctly predicted positive **TN (True Negative):** Correctly predicted negative **FP (False Positive):** Wrongly predicted positive (Type I error) **FN (False Negative):** Wrongly predicted negative (Type II error)

💡 Analogy: Fire Alarm System

```
              Alarm Rings    Alarm Silent
Actual Fire   |  TP ✅      |  FN ⚠️     | ← Missed fire!
No Fire       |  FP 😤      |  TN ✅     |
              False          True
              Alarm          Silence
```

- **TP:** Fire exists, alarm rings → Good!
- **TN:** No fire, alarm silent → Good!

- **FP:** No fire, but alarm rings → Annoying but safe (false alarm)
- **FN:** Fire exists, but alarm silent → DANGEROUS! (missed detection)

Key insight: FP and FN are NOT equally bad. Context matters!

💡 Another Analogy: Medical Test for Disease

```
                Test Positive   Test Negative
Has Disease  |    TP ✅     |    FN ⚠️     | ← Missed diagnosis!
Healthy      |    FP 😰     |    TN ✅     |
                False          Healthy
                Positive       (correctly cleared)
```

- **FN (False Negative):** Patient has cancer, test says healthy → CRITICAL MISS
- **FP (False Positive):** Patient healthy, test says cancer → Unnecessary stress/treatment

Which is worse depends on context:

- Cancer screening: FN worse (missing cancer is deadly)
- COVID screening at airport: FP more acceptable (false alarm, but safe)

🎯 Accuracy

Formula: (TP + TN) / (TP + TN + FP + FN) **Meaning:** % of all predictions that were correct

💡 **Analogy:** Exam grade

> Out of 100 questions, you got 85 right → 85% accuracy. Simple, intuitive.

When it's misleading: Imbalanced datasets

Example: Credit card fraud detection

- 99% of transactions are legitimate
- 1% are fraudulent

Naive model: Predict "legitimate" for everything

- Accuracy = 99%! Looks great!
- But catches 0% of fraud! Useless!

Rule: Only use accuracy when classes are balanced (~50/50)

🎯 Precision

Formula: TP / (TP + FP) **Meaning:** Of all positive predictions, how many were actually correct?

💡 Analogy: Email spam filter

> Filter marks 100 emails as spam. You check → only 80 were actually spam, 20 were important work emails wrongly flagged.
> Precision = 80/100 = 80%
> **Question precision answers:** "When the model says SPAM, how often is it right?"

Use when: False Positives are costly

- Spam filter (don't want important emails filtered)
- Innocent person convicted of crime
- Recommending a bad product to a customer

🎯 Recall (Sensitivity / True Positive Rate)

Formula: TP / (TP + FN) **Meaning:** Of all actual positives, how many did we catch?

💡 Analogy: Airport security metal detector

> 100 people carrying metal objects pass through detector.
> Detector beeps for 75 of them, misses 25.
> Recall = 75/100 = 75%
> **Question recall answers:** "Of all the actual threats, how many did we detect?"

Use when: False Negatives are costly

- Cancer screening (missing cancer is deadly)
- Fraud detection (missing fraud costs money)
- Search engine (missing relevant results frustrates users)

🎯 Precision vs Recall Tradeoff

💡 Analogy: Fishing with a net

High Precision, Low Recall:

> Tiny net with very fine mesh. Every fish you catch is the exact species you want (high precision), but you miss most of the fish in the lake (low recall). You're picky.

Low Precision, High Recall:

> Giant net with huge holes. You catch almost all the target fish (high recall), but also catch lots of seaweed, boots, and wrong fish (low precision). You're not picky.

The tradeoff:

- Make threshold stricter (higher confidence needed) → Precision ↑, Recall ↓
- Make threshold looser (lower confidence accepted) → Precision ↓, Recall ↑

Real Example: Email Spam Filter

Conservative filter (high precision):

- Only flags email as spam if 95% confident
- Precision: 95% (rarely wrong when it flags spam)
- Recall: 60% (misses 40% of spam)
- **Result:** Inbox has some spam, but no important emails lost

Aggressive filter (high recall):

- Flags email as spam if 50% confident
- Precision: 70% (often wrong, flags real emails as spam)
- Recall: 95% (catches almost all spam)
- **Result:** Clean inbox, but important emails lost in spam folder

Your choice depends on what matters more to you!

🎯 F1-Score

Formula: 2 × (Precision × Recall) / (Precision + Recall) **Meaning:** Harmonic mean of precision and recall

💡 **Analogy:** Balancing speed and accuracy in a race

> Runner A: Fast but sloppy (high recall, low precision)
> Runner B: Slow but accurate (high precision, low recall)
> Runner C: Balanced (F1 score optimizes both)
> F1 score punishes imbalance — you can't game it by being extreme in one direction.

When to use: When you care about BOTH precision and recall equally

- Medical diagnosis (don't want to miss disease OR scare healthy people)
- Fraud detection (catch fraud without annoying legitimate customers too much)

Why harmonic mean, not arithmetic?

```
Precision = 100%, Recall = 10%

Arithmetic mean = (100 + 10) / 2 = 55%  ← Looks okay!
Harmonic mean (F1) = 2×(100×10)/(100+10) = 18%  ← Punishes imbalance!

F1 score forces you to balance both metrics.
```

🎯 ROC Curve & AUC

ROC Curve (Receiver Operating Characteristic)

What: Plot of True Positive Rate (Recall) vs False Positive Rate at different thresholds

```
    TPR
  (Recall)
    ↑
  1 |    ___----Perfect
    |   /
    |  /
  0.5  Random
    |/
    |_____________→
    0     0.5     1
          FPR
```

💡 **Analogy:** Metal detector sensitivity dial

> Airport metal detector has a sensitivity dial:
>
> - Low sensitivity: Only beeps for large metal (guns) → Low FPR, Low TPR (misses small weapons)
> - Medium sensitivity: Beeps for guns and knives → Medium FPR, High TPR (some false alarms)
> - High sensitivity: Beeps for everything including keys, belt buckles → High FPR, High TPR (too many false alarms)
>
> ROC curve plots performance across ALL possible sensitivity settings.
> AUC = area under this curve = how good is the detector across all settings?

AUC (Area Under the Curve)

Range: 0 to 1 **Meaning:** Probability that model ranks a random positive higher than a random negative

AUC	Interpretation
1.0	Perfect classifier
0.9-1.0	Excellent
0.8-0.9	Good
0.7-0.8	Fair
0.5	Random (coin flip)
<0.5	Worse than random

💡 **Analogy:** Sorting quality

> You have 100 resumes — 50 qualified, 50 unqualified. Model scores each resume.
> AUC = 1.0: Model gives ALL qualified candidates higher scores than ALL unqualified → Perfect sorting
> AUC = 0.5: Model's scores are random, no better than drawing names from a hat
> AUC = 0.8: Model usually ranks qualified higher, but makes some mistakes

When to use:

- Comparing models across different thresholds
- Imbalanced datasets (AUC is less sensitive to imbalance than accuracy)
- When you need a single number to compare models

🎯 Precision-Recall Curve

What: Plot of Precision vs Recall at different thresholds **When to use:** Heavily imbalanced datasets (better than ROC in this case)

💡 **Analogy:** Quality vs Quantity in search results

> You search for "best restaurants in Chennai"
> - High precision, low recall: First 5 results are perfect, but you miss 95% of good restaurants
> - Low precision, high recall: All 100 results shown but 80 are bad
> - PR curve shows this tradeoff at every threshold

PART 3 — REGRESSION METRICS

🎯 MAE (Mean Absolute Error)

Formula: $(1/n) \times \Sigma|\text{actual} - \text{predicted}|$ **Meaning:** Average absolute difference between prediction and actual

💡 **Analogy:** Estimating travel time

> You estimate trip takes 60 minutes.
> Actual times: 55 min, 70 min, 58 min, 65 min
> Errors: 5, 10, 2, 5
> MAE = (5+10+2+5)/4 = 5.5 minutes
> On average, your estimates are off by 5.5 minutes.

Units: Same as target variable (easy to interpret!)

Properties:

- Treats all errors equally (error of 10 = twice as bad as error of 5)
- Robust to outliers (doesn't square errors)

Use when: You care about average error and outliers shouldn't dominate

---

🎯 MSE (Mean Squared Error)

Formula: $(1/n) \times \Sigma(\text{actual} - \text{predicted})^2$ **Meaning:** Average squared difference

💡 **Analogy:** Grading system with extra penalty for big mistakes

> Student makes errors: 1 point, 1 point, 10 points
> MAE = (1 + 1 + 10) / 3 = 4 points average
> MSE = $(1^2 + 1^2 + 10^2) / 3 = (1 + 1 + 100) / 3 = 34$
> MSE punishes that one big 10-point error much more heavily (100 vs 10)

Properties:

- Penalizes large errors heavily (squared term)
- Sensitive to outliers
- Units: squared units (harder to interpret)

Use when: Large errors are particularly bad (e.g., stock predictions — being off by $100 is more than 10× worse than being off by $10)

🎯 RMSE (Root Mean Squared Error)

Formula: $\sqrt{\text{MSE}}$ **Meaning:** Square root of MSE → back to original units

💡 **Analogy:** MSE in understandable units

> House price prediction
> MSE = 4,000,000,000 (4 billion — hard to interpret)
> RMSE = 63,245 (sixty-three thousand — "predictions off by ~$63k on average")

Properties:

- Same properties as MSE but in original units
- Still penalizes large errors more than MAE
- Most commonly used regression metric

Use when: You want MSE's outlier sensitivity but need interpretable units

---

🎯 R^2 (R-Squared / Coefficient of Determination)

Formula: 1 - (SS_residual / SS_total) **Range:** -∞ to 1 (usually 0 to 1) **Meaning:** % of variance in target variable explained by model

💡 Analogy: How much does studying explain test scores?

```
Students' test scores vary from 40 to 100.

Scenario 1: Predict everyone gets 70 (mean)
→ Captures 0% of variance → R² = 0

Scenario 2: Predict based on hours studied
→ Predictions: 45, 65, 85, 95
→ Captures 80% of variance → R² = 0.8

Scenario 3: Perfect predictions
→ Captures 100% of variance → R² = 1.0
```

R^2	Meaning
1.0	Perfect predictions (100% of variance explained)
0.8	Good (80% of variance explained)
0.5	Moderate (50% of variance explained)
0	Useless (no better than predicting mean)
<0	Worse than predicting mean (terrible model)

💡 **Another Analogy:** Weather prediction

> How well can you predict temperature?
> $R^2 = 0$: Predict "average temperature" every day → captures no variation
> $R^2 = 0.6$: Use season/month → explains 60% of why temps vary
> $R^2 = 0.9$: Use season + pressure + humidity → explains 90% of variation

Use when: You want to know "how good is my model overall?" in a 0-1 scale

---

🎯 When to Use Which Regression Metric

Metric	Use When	Example
MAE	Want interpretable error, outliers shouldn't dominate	Delivery time estimation
RMSE	Large errors are especially bad	House price prediction
R^2	Want to know % of variance explained	Model comparison

💡 **Rule of thumb:** Report all three — they tell different stories!

PART 4 — UNSUPERVISED LEARNING METRICS

🎯 Clustering Evaluation

Silhouette Score

Range: -1 to 1 **Meaning:** How well-separated are the clusters?

Formula: (b - a) / max(a, b)

- a = average distance to points in same cluster
- b = average distance to points in nearest other cluster

💡 **Analogy:** Social cliques at a party

> Good clustering (Silhouette = 0.8):
> - You stand close to your friend group (small a)
> - Far from other groups (large b)
> - Clear, distinct groups
>
> Bad clustering (Silhouette = 0.2):
> - Friend group is scattered (large a)
> - Other groups are close by (small b)

> - Groups overlap, hard to tell apart

Score	Meaning
0.7-1.0	Strong, well-separated clusters
0.5-0.7	Reasonable structure
0.25-0.5	Weak, overlapping clusters
<0.25	No meaningful structure

Inertia (Within-Cluster Sum of Squares)

What: Sum of squared distances from each point to its cluster center **Lower = better** (points are close to their cluster centers)

💡 **Analogy:** Student dormitory assignments

> University assigns students to dorms based on major.
> Low inertia: Engineering students in Engineering dorm, Arts students in Arts dorm → students walk short distances to classes
> High inertia: Random assignment → students walk long distances, no logical grouping

Problem: Always decreases as k increases (k=n gives inertia=0) **Solution:** Use elbow method → plot inertia vs k, find the "elbow" where it stops improving much

---

Davies-Bouldin Index

Range: 0 to ∞ (lower is better) **Meaning:** Average similarity between each cluster and its most similar cluster

💡 **Analogy:** City neighborhoods

> Low DB Index: Each neighborhood has distinct character (Italian, Chinese, Business district) → easy to tell apart
> High DB Index: Neighborhoods blur together → hard to say where one ends and another begins

---

🎯 Dimensionality Reduction Evaluation

Explained Variance Ratio (PCA)

What: % of total variance captured by each principal component

💡 **Analogy:** Describing a movie plot

> Movie has 100 scenes (features).
> PC1 captures 60% of variance: "It's a love story" → explains most of what happens
> PC2 captures 20%: "With a heist subplot" → adds important detail
> PC3 captures 10%: "Set in Paris" → minor detail
> PC4-100 capture 10%: Tiny details you can ignore
> Using PC1+PC2 (80% variance) gives you the gist without all 100 scenes

Rule: Keep components until you hit 90-95% cumulative variance

---

Reconstruction Error

What: Difference between original data and data reconstructed from lower dimensions

💡 **Analogy:** Photo compression (JPEG)

> Original photo: 10 MB (high-dimensional)
> Compressed: 1 MB (low-dimensional)
> Reconstruct: Looks almost the same but some quality lost
> Reconstruction error = how much quality you lost in compression

Lower = better (means dimensionality reduction preserved information well)

PART 5 — PRACTICAL DECISION GUIDE

🎯 "Which Metric Should I Use?" Decision Tree

For Classification:

```
Do you have balanced classes?
├── Yes → Use Accuracy (simple, interpretable)
└── No (imbalanced) → Continue...
    │
    What matters more?
    ├── Missing positives is costly (cancer, fraud)
    │   └── Optimize for Recall
    │
    ├── False alarms are costly (spam filter, criminal justice)
    │   └── Optimize for Precision
    │
    ├── Both matter equally
    │   └── Optimize for F1-Score
    │
```

```
    └── Need threshold-independent comparison
        └── Use AUC-ROC (balanced) or PR-AUC (very imbalanced)
```

For Regression:

```
Are there outliers in your data?
├── No → Use RMSE (standard choice)
├── Yes, and they're errors to ignore
│   └── Use MAE (robust to outliers)
└── Yes, and large errors are especially bad
    └── Use RMSE (penalizes outliers)

Also always report R² for overall model quality (0-1 scale)
```

For Clustering:

```
Do you know the true labels?
├── Yes (rare) → Use Adjusted Rand Index or Normalized Mutual Information
└── No (common) → Use Silhouette Score + visual inspection

Also check:
- Elbow method (inertia vs k)
- Silhouette plot (per-cluster quality)
- Domain knowledge (do clusters make sense?)
```

🎯 Quick Reference Table

Classification

Metric	Formula	Range	Good Value	Use Case
Accuracy	(TP+TN)/Total	0-1	>0.9	Balanced classes
Precision	TP/(TP+FP)	0-1	>0.8	Minimize false positives
Recall	TP/(TP+FN)	0-1	>0.8	Minimize false negatives
F1	2PR/(P+R)	0-1	>0.8	Balance P and R
AUC-ROC	Area	0-1	>0.8	Threshold-independent

Regression

Metric	Units	Lower/Higher Better	Interpretable?
MAE	Same as y	Lower	✅ Yes
RMSE	Same as y	Lower	✅ Yes
MSE	y^2	Lower	❌ No
R^2	None	Higher (max 1)	✅ Yes

Clustering

Metric	Range	Good Value	What it measures
Silhouette	-1 to 1	>0.5	Cluster separation
Inertia	0 to ∞	Lower	Compactness
Davies-Bouldin	0 to ∞	Lower	Inter/intra cluster ratio

🎯 Practice Questions

1. Your cancer screening model has 95% accuracy but 20% recall. Is this good?
2. Why is F1-score the harmonic mean and not arithmetic mean?
3. When would you prefer MAE over RMSE?
4. Explain bias-variance tradeoff using a real-world example
5. Your model has $R^2 = 0.3$. Is it useful?
6. What's the difference between precision and recall in one sentence each?
7. Why can't you use accuracy for fraud detection (1% fraud rate)?
8. What does Silhouette score = 0.1 tell you about your clustering?

Remember: Metrics are tools. The "best" metric depends on your problem, not universal rules. Always ask: "What mistake is most costly in THIS specific situation?"